AURIX's Developer Role Guide

🌐 Role : Backend & Infrastructure Engineer

Assigned to: BHAVYA

1. The "Why" of Your Component

You are the "Traffic Cop," the "Vault," and the Nervous System of AURIX. The Web UI and VS Code extension cannot run heavy AI models directly, nor can they communicate directly with each other. Your component securely receives user code, authenticates users, stores data, queues it up, and orchestrates the communication between the client interfaces and the heavy AI Worker Node. If your API goes down or is breached, the entire product fails.

2. Your Domain in the Architecture (HLD & LLD References)

As the Backend Engineer, you are the bridge holding the system together. You must refer strictly to the following architectural blueprints:

- **HLD Ownership:** You own **Layer 2 (Backend API & Storage Gateway)** in the aurix-HLD.mmd or png diagram. This includes the API Server, Primary DB, S3 Storage, and the Redis Task Queue.
- **LLD Ownership (Database & Auth):** You are responsible for implementing the exact SQL schema defined in the database-er-diagram.mmd or png. You must create the PROJECTS, SCANS, and VERIFIED_VULNERABILITIES tables exactly as mapped. Furthermore, you own the Users data via Supabase Auth.
- **LLD Ownership (API Flows):** You own the backend side of the HTTP requests defined in **both** API sequence diagrams:
  - webDashboard-api-sequence-diagram.mmd or png : Handling POST /api/scans/github and GET /api/scans/{scan_id}.
  - Vscode-extension-api-sequence-diagram.mmd or png : Handling POST /api/scans/upload (multipart/form-data for zip files).

3. Step-by-Step Guide

1. **Provision Infrastructure:** Set up the free tiers of Supabase (for PostgreSQL, Cloud Storage, and Authentication) and Upstash (for serverless Redis).
2. **Setup the "Vault" (Authentication):** Configure Supabase Auth. Build the API routes that receive login credentials from the Frontend, validate them, and issue secure session tokens (JWTs) so users can only view their own scan reports and projects.
3. **Database Schema:** Execute the SQL to create the core scan and project tables according to your ERD diagram, ensuring they are linked securely via Foreign Keys to the authenticated user's ID.
4. **Build the REST API Gateway:** Initialize a FastAPI (Python) or Express (Node.js) server and

deploy it to a free host like Render or Railway.

5. **Implement the Core Routes:**
    - POST /api/scans/github: Save the project to the DB, set status to PENDING, push the URL to Redis, and return 202 Accepted.
    - POST /api/scans/upload: Receive the .zip file, upload it to Supabase Storage, create the DB record, push the storage link to Redis, and return 202 Accepted.
    - GET /api/scans/{scan_id}: Look up the scan in the database and return its current status and/or findings.
6. **Implement the Webhook:** Create an internal route that the AI Worker Node (Divyansh) will call to POST the final verified bugs and update the scan status to COMPLETED.
7. **Automated Data Sanitization:** Implement a data retention policy using a Cron job or storage TTL (Time To Live). Once a scan is marked COMPLETED or FAILED, the backend must automatically delete the .zip payload from Supabase Storage.
8. **Rate Limiting & Cloud Compute Quotas:** Implement Redis-backed API Rate Limiting. Restrict users to a maximum number of scans per hour/day based on their account tier, returning a 429 Too Many Requests status if they exceed the limit.

4. Expected Output / Streamlining Flow

A deployed, publicly accessible API Gateway that reliably handles user logins, accepts scan requests, instantly returns a scan_id (without making the user wait for the AI to finish), successfully queues the job in Redis, and provides a stable endpoint for the frontends to poll for status updates.Furthermore, the system will autonomously manage storage limits via sanitization and protect cloud compute resources through strict rate limiting.

5. Critical "Do Not Miss" Constraints

- **Authentication Handshake:** You must supply the Frontend Engineer with the exact API routes needed for login/signup, and enforce authorization checks on all GET and POST routes to prevent data leaks.
- **Asynchronous Processing is Mandatory:** *Never* make your HTTP POST request wait for the AI scan to finish. The AI takes minutes to run; HTTP requests time out after 30 seconds. Accept the request, push to Redis, save to DB, and immediately return a 202 Accepted response.
- **Robust Zip File Handling:** The /upload endpoint must safely handle file streams of ~10MB zip files, uploading them directly to Supabase Storage without crashing your lightweight Render server's memory.
- **Data Privacy :** You only need to keep the VERIFIED_VULNERABILITIES metadata long-term, not the source code itself. Ensure the .zip code files are deleted reliably post-scan to prevent massive storage bloat and data breach targets.
- **Compute Protection :** Do not allow runaway scripts or malicious users to flood the queue. Rate limiting must block excess requests before they hit the database or Redis queue.

6. Prerequisites & Skills Needed

- REST API Design (FastAPI or Node/Express).
- Relational Databases and SQL (PostgreSQL).
- Authentication Protocols (JWT, Session Management).
- Handling Cloud Object Storage (Supabase/S3) and multipart form data.
- Message Queues / Pub-Sub systems (Redis).

7. Reference Materials

Docs provided by tech leads

- aurix-HLD.mmd or png
- database-er-diagram.mmd or png
- webDashboard-api-sequence-diagram.mmd or png
- Vscode-extension-api-sequence-diagram.mmd or png

External documentation

- Supabase Auth & Database Docs
- Upstash Redis Quickstart
- FastAPI or Express.js Official Documentation.

8. Component Build List & Utility

- **API Gateway:** The central router that allows the Web and VS Code clients to interact with the system.
- **The Auth Vault:** Manages user identities, issues tokens, and protects private scan data.
- **PostgreSQL Database:** The persistent memory of the system, storing the history and results of all scans.
- **Redis Queue:** The shock absorber that prevents the AI engine from crashing if multiple users request a scan at the exact same time.
- **Automated Data Sanitizer :** A background task that cleans up raw user code from storage to minimize security risks and cost overhead.
- **Rate Limiter:** A Redis-based middleware that enforces usage quotas to protect computationally expensive AI resources.

9. 🚨 CRITICAL DEPENDENCIES (You are the Hub)

- **You BLOCK the Frontend (Bhumika) and VS Code (Divyanshi) Engineers:** They cannot build their UI polling logic or their login screens without your API endpoints. **Your very first task** must be to create "Mock Endpoints" (fake routes that return hardcoded JSON for both scans and authentication) and deploy them so your teammates can start building their UI immediately while you wire up the real database.
- **You are BLOCKED BY the Core Engine Architect (Divyansh):** You need Divyansh to finalize the exact JSON structure of the verified_reports output from the LangGraph

agents before you can finalize how you will save that data into your VERIFIED_VULNERABILITIES table.